

## Laboratorio 2 –Tratamiento de “arrays”

En esta sesión se trabajará la gestión de arrays en Java y varios algoritmos de ordenación de arrays. Además, de cara a contestar las preguntas realizadas en este guion, será necesario utilizar el depurador.

### Objetivos

Los objetivos específicos de esta sesión son:

- Entender cómo funciona la declaración y manipulación de arrays en Java.
- Ser capaz de implementar un algoritmo de ordenación de *arrays*.
- Practicar de forma intensiva el uso del depurador para resolver problemas de diseño de programas.

### Arrays de valores de tipo primitivo

Una buena parte de esta práctica la destinaremos a trabajar con *arrays*. Lo que sigue es una breve explicación de cómo funcionan y las peculiaridades de la sintaxis Java para su gestión, que es muy parecida a la sintaxis C, salvo en algunos detalles.

Un **array** en Java es una variable que permite almacenar en forma de secuencia, una cantidad predeterminada:

- O bien valores que pertenecen al mismo tipo primitivo de datos (int, double, boolean, etc.), tal y como muestra la Figura 1. Cada valor ocupa una posición concreta dentro del array.
- O bien objetos instancias de una misma clase.

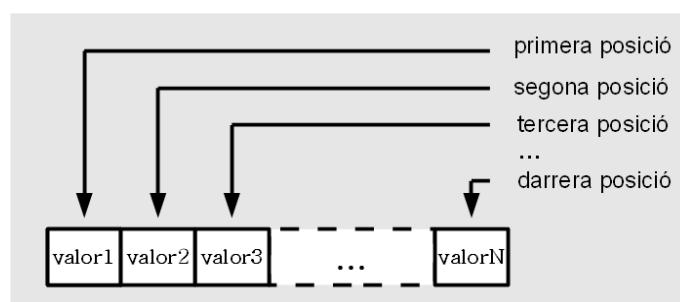


Figura 1. Estructura de un array

Cuando un array contiene valores de tipo primitivo, la forma más sencilla de declararlo y crearlo es enumerando los valores que deben estar presentes en dicho array. El vector creado tendrá una dimensión igual al número de valores presentes en la lista. El código que sigue muestra la declaración, creación y llenado (en una sola instrucción) de un array de dimensión 10 con los valores int mostrados en la lista.

```
/*"arrayEnteros" es el nombre de la variable. Observad la sintaxis de la lista: valores separados por  
comas encerrados entre { y }  
*/  
int [] arrayEnteros = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
```

Cuando se conoce a priori la dimensión del vector pero no los valores que contendrá, se puede declarar y crear el array primero, y luego asignar valores a sus posiciones conforme dichos valores sean conocidos.

El código que sigue muestra cómo declarar y crear un array en una instrucción.

```
int [] arrayEnteros = new int[10];
```

La instrucción crea un array de dimensión 10. Puesto que la instrucción no establece valores para las diferentes posiciones del array, a esas posiciones se les asigna unos VALORES POR DEFECTO, según las siguientes reglas:

1. Para valores numéricos tal valor es el 0 (para tipos primitivos que gestionan números enteros) o el 0.0 (para tipos primitivos que gestionan números reales).
2. Para valores booleanos el valor por defecto es false.
3. Para caracteres el valor por defecto es el carácter con todos sus bits puestos a 0.

Debéis tener en cuenta que, al igual que sucede en C, la variable que gestiona un array en Java es un puntero, es decir, su valor es la dirección de la posición de memoria en la que comienzan a almacenarse los valores presentes en el array.

En consecuencia la sentencia anterior creará un array de dimensión 10 cuyas 10 posiciones contendrán el valor entero 0. La dimensión de un array puede ser también el valor de una variable entera, como se muestra en el código que sigue:

---

```
int numPos = 10 ;  
  
int [] arrayEnteros = new int[numPos];
```

---

Finalmente, la declaración y la creación del array pueden realizarse en instrucciones diferentes, de tal manera que entre la declaración y la creación puede calcularse la dimensión de dicho array. Ello puede traducirse en que la dimensión del vector pueda cambiar de una ejecución a otra.

---

```
/* El array se declara PERO NO se crea. Tras la ejecución de esta instrucción NO existe un array en memoria. */  
  
int [] arrayEnteros, numPos;  
  
// Aquí vendría código en el que se asigna un valor a la variable numPos  
  
/* La que sigue es la instrucción que crea el vector con una dimensión igual al valor de la variable numPos y asigna a todas sus posiciones el valor por defecto (0 en este caso). No hay impedimento alguno para que el código de asignación de valor a la variable numPos sea tal que ejecuciones diferentes resulten en valores diferentes: ello permite que en cada ejecución la dimensión del array sea la estrictamente necesaria. Tal cosa no podía hacerse en C usando sintaxis de gestión de arrays */  
  
arrayEnteros = new int[numPos];
```

---

En un array Java el índice de la primera posición es 0, y el de la última es su dimensión menos 1.

El código siguiente deposita en la variable `dim` la dimensión del vector.

---

```
int [] arrayEnteros = new int[10];  
  
int dim = arrayEnteros.length ;
```

---

Cuando se ha creado el array, la sintaxis de gestión (usando corchetes) de las posiciones del mismo es idéntica a la de C.

El siguiente código muestra el acceso a la posición 3 de un array de enteros:

---

```
int [] arrayEnteros = new int[10];  
  
arrayEnteros[3] = 6 ;  
  
int val = arrayEnteros[3] ;
```

---

Los programas Java son ejecutados por otro programa: la máquina virtual de Java. A diferencia de lo que ocurre en C, el intento de acceso a una posición de un array que no existe es SIEMPRE detectada por la máquina virtual. Ante estas situaciones, la máquina virtual reacciona notificando la aparición de esta situación excepcional. Si el código del programa no tiene nada previsto para tales casos, la máquina virtual aborta la ejecución del programa.

---

```
int [] arrayEnteros = new int[10];  
  
/* ¡¡ERROR DE DISEÑO!!: los índices del array varían entre 0 y 9. La posición con índice 10 NO EXISTE en el vector anterior. La máquina virtual reaccionará notificando la aparición de una situación excepcional  
  
*/  
  
arrayEnteros[10] = 6 ;
```

---

Si se intenta ejecutar el anterior código, la máquina virtual aborta la ejecución y presenta un mensaje parecido al que sigue:

---

```
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: 8  
    at ac.upc.edu.sort.InsertionSort.main(InsertionSort.java:55)
```

---

Observad que se notifica `ArrayIndexOutOfBoundsException`, esto es, una situación excepcional consistente en que hay un índice que se halla fuera de los límites de la dimensión del array.

La línea de debajo muestra al usuario en qué línea de qué archivo del programa se forzó la aparición de esa situación excepcional.

Con esta información, el diseñador sabe dónde debe fijar su atención para tratar de solucionar el problema de diseño.

## Arrays de objetos

Los arrays pueden contener también objetos. La sintaxis de gestión de estos arrays es similar a la de los arrays de valores de tipos primitivos, tal y como muestra el código que sigue:

---

```
/* Declaración y creación por listado de valores (objetos) de un vector de objetos Integer de dimensión 3. OBSERVAD QUE LA LISTA CONTIENE OBJETOS CREADOS EN LA MISMA LÍNEA*/
```

---

```
Integer[] arrayObjs1 = {new Integer(1), new Integer(2), new Integer(3)} ;  
/* OBSERVAD AHORA QUE LA LISTA PUEDE INCLUIR VARIABLES QUE REFERENCIAS A OBJETOS CREADOS ANTES */  
Integer a1 = new Integer(1), a2 = new Integer(2) ;  
Integer[] arrayObjs2 = {a1, a2} ;
```

---

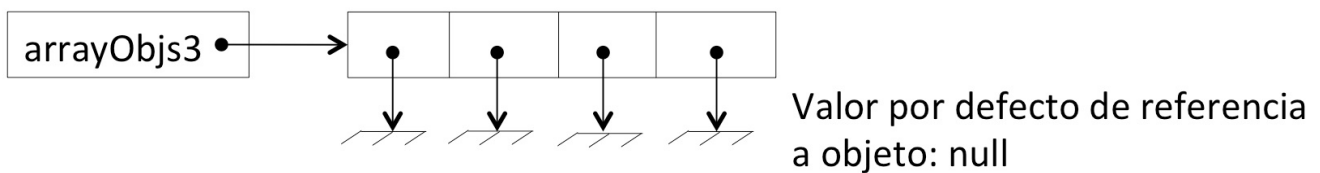
Algo que hay que tener siempre muy presente ES QUE **LAS POSICIONES DE LOS ARRAYS DE OBJETOS NO CONTIENEN OBJETOS, SINO REFERENCIAS A OBJETOS.**

---

```
/* DECLARACIÓN Y CREACIÓN SEPARADAS: la primera instrucción NO CREA NINGÚN ARRAY DE OBJETOS. La segunda  
instrucción CREA UN ARRAY CAPAZ DE GESTIONAR 4 OBJETOS Integer */  
Integer[] arrayObjs3 ;  
arrayObjs3 = new Integer[4] ;
```

---

La figura que sigue muestra el efecto neto en memoria de la ejecución de las dos instrucciones del código anterior:



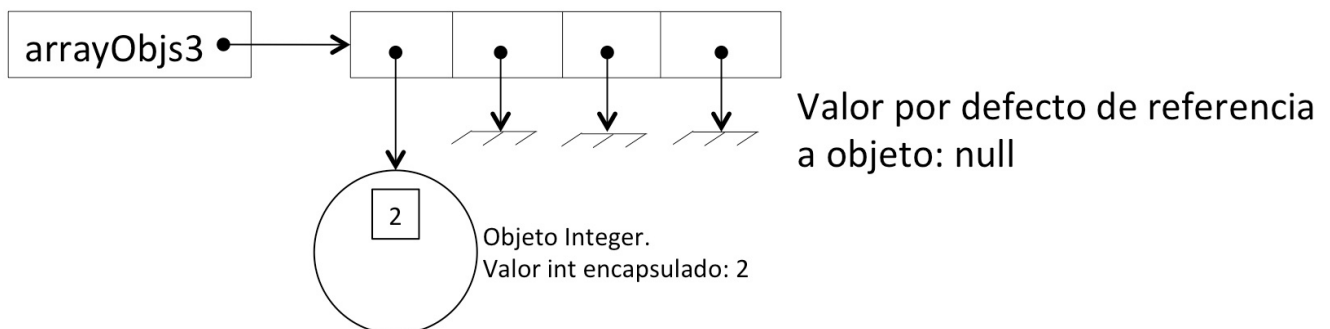
Como en los arrays de valores de tipos primitivos, la segunda instrucción crea un array de dimensión 4, pero sus posiciones contiene REFERENCIAS a objetos. Como la instrucción de creación no asigna valores a las posiciones, éstas toman los valores por defecto. El valor por defecto de toda referencia a un objeto es la referencia de valor null.

Ahora podemos crear objetos Integer y depositar referencias a dichos objetos en las posiciones del array como se muestra en el siguiente código.

```
/* La siguiente instrucción crea un objeto Integer y lo "asigna" a la posición 0 del array*/  
arrayObjs3[0] = new Integer(2);
```

---

La figura que sigue muestra el efecto en memoria de la ejecución de dicha instrucción:



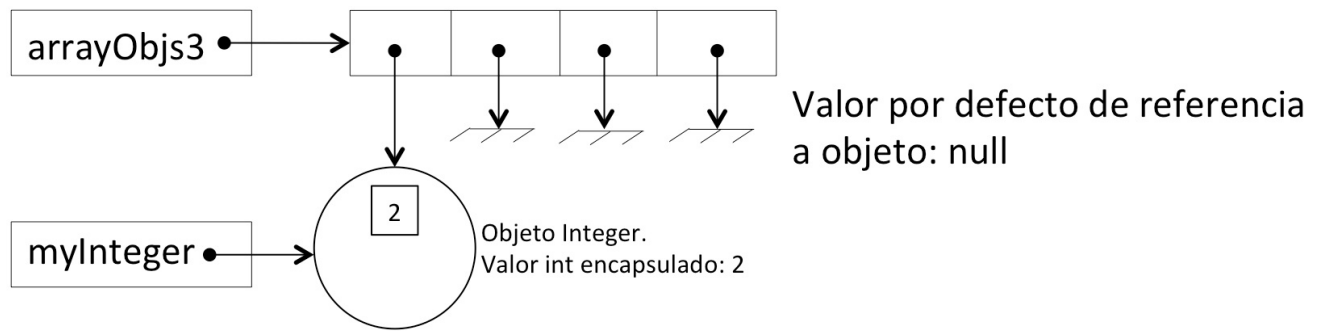
Observad que la sintaxis de asignación de valor a la posición 0 del array conlleva que en dicha posición aparezca una referencia al objeto Integer.

Igualmente podemos hacer que una variable referencia a un objeto Integer tome el valor de la referencia existente en la posición 0

```
/* La siguiente instrucción hace que la variable myInteger referencie al objeto Integer que ocupa la  
posición 0 del array */  
Integer myInteger = arrayObjs3[0] ;
```

---

La figura que sigue muestra el efecto en memoria de la ejecución de esta instrucción:



## Arrays multidimensionales

Java, como C, permite definir arrays multidimensionales. A los bidimensionales se les denomina también matrices. A continuación se muestra código para la gestión de arrays multidimensionales.

```
/* Esta línea declara y crea un array bidimensional (matriz) de 3 filas por 2 columnas. Observad cómo se generaliza la creación por enumeración de valores. Los valores de cada fila aparecen encerrados entre { y } */
int[][] mat1 = {{1,2},{3,4},{5,6}} ;

/* Esta línea muestra algo que en C no puede hacerse usando sintaxis de arrays: tener arrays multidimensionales en los que cada "fila" tiene una dimensión diferente. La primera fila tiene dimensión 2, la segunda 3 y la tercera 4 */
int[][] mat2 = {{1,2},{3,4,5},{5,6,7,8}} ;

/* La primera línea declara un array bidimensional; la segunda crea uno de 3 filas por 4 columnas. Al ser de valores int, todas las posiciones toman el valor 0 */
int[][] mat3 ;

mat3 = new int[3][4] ;

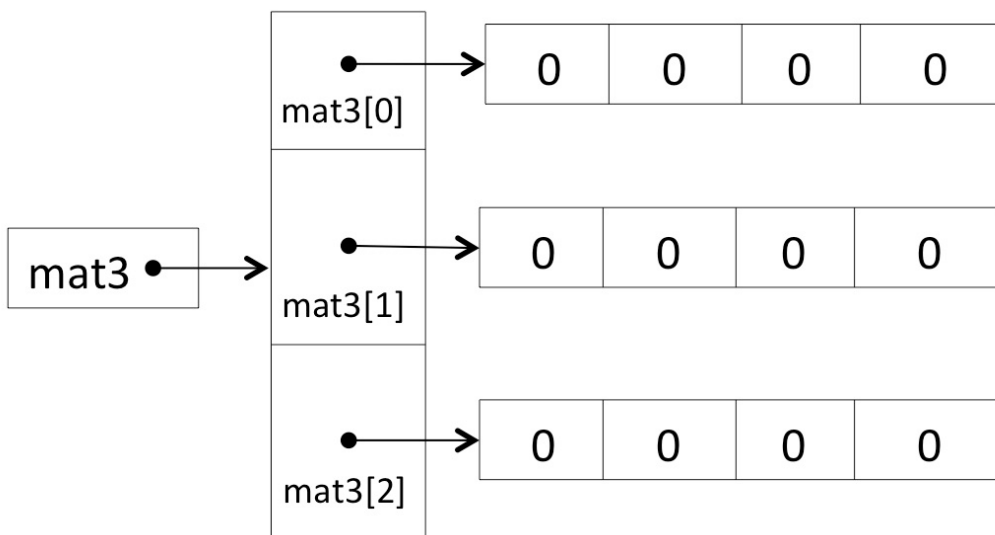
/* La primera línea declara un array tridimensional; la segunda crea uno de dimensión 2 en la primera, 3 en la segunda y 4 en la tercera. Al ser de valores int, todas las posiciones toman el valor 0 */
int[][][] trid1 ;

trid1 = new int[2][3][4];

/* Las instrucciones que siguen muestran que el acceso a los contenidos de las posiciones de un array mutidimensional en Java se realiza igual que el acceso a los contenidos de las posiciones de un array multidimensional en C, usando los índices entre corchetes [] */
mat3[0][0] = 2 ;

int a = mat3[0][0] ;
```

La figura que sigue muestra el efecto en memoria de la ejecución de las dos instrucciones que crean un array bidimensional de valores int 3 por 4.



Observad que un array bidimensional es un array de arrays. Observad cómo la variable `mat3` es una referencia a un array cuyas posiciones son a su vez referencias a arrays.

Lo anterior es válido para arrays multidimensionales de objetos, con la salvedad de que ahora las posiciones

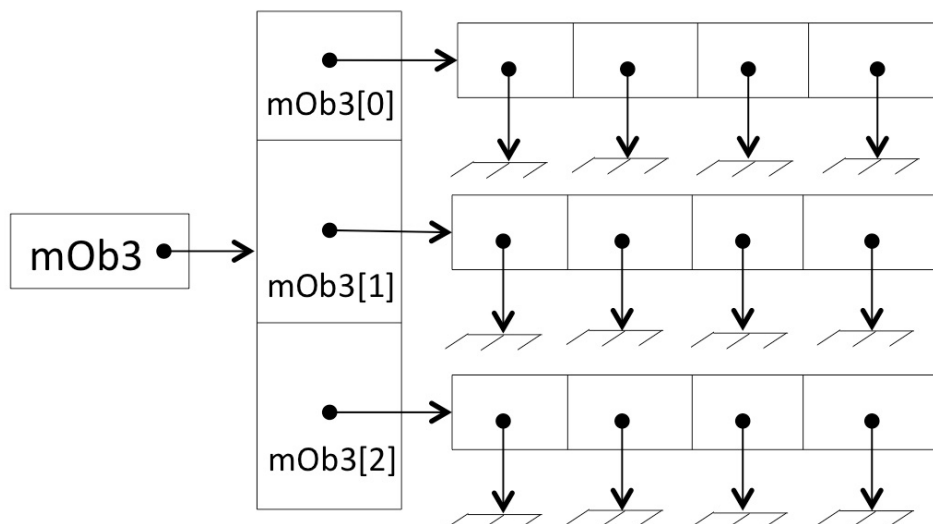
de los arrays contienen referencias a objetos, tal y como se ha mencionado antes.

```
/* La primera línea declara un array bidimensional de objetos Integer; la segunda crea uno de 3 filas por 4 columnas. Al ser de objetos Integer, todas las posiciones toman el valor null */
```

```
Integer [][] mOb3 ;
```

```
mOb3 = new Integer [3][4] ;
```

La figura que sigue muestra el efecto en memoria de la ejecución de las dos instrucciones que crean un array bidimensional de objetos Integer por 4.



Observad cómo ahora los 3 arrays de 4 posiciones donde se guardan las referencias a los objetos toman el valor null.

## Tareas a realizar

En lo que sigue se os propondrá programar una serie de algoritmos de ordenación de arrays cuyos contenidos se conocen de antemano. Después se os pedirá que modifiquéis vuestro código para llenar los arrays con valores aleatorios.

### Apartado 1: Ordenación por el método de inserción directa (“direct insertion”)

Este método es usado en muchas ocasiones por los jugadores de cartas.

Principios del método de ordenación:

El método **realiza N-1 recorridos parciales por el array, siendo N la dimensión del array.**

En cada uno de ellos el algoritmo toma el valor del elemento *i*-ésimo del array como valor de referencia. **El primer recorrido toma como valor de referencia el valor de la segunda posición del array (posición con índice 1).** Los siguientes recorridos toman los valores de las posiciones *i*+1, *i*+2, ..., N como valores de referencia.

En cada recorrido **se inspeccionan los valores que quedan a la izquierda de la posición *i*** que ocupa el valor de referencia y se genera un hueco en una determinada posición *h* (menor o igual que *i*) desplazando hacia la derecha todos los valores que son mayores que el valor de referencia. Al final del recorrido los valores entre las posiciones 0 y *h*-1 serán menores que el valor de referencia y los valores entre las posiciones *h*+1 e *i* serán mayores que el propio valor de referencia. Así que la zona del array comprendida entre las posiciones 0 e *i* será una zona de valores ordenados. Cada nuevo recorrido ampliará la zona ordenada en una posición. Al finalizar el último, el vector estará ordenado.

Algoritmo de ordenación de un array *a* de longitud *N*:

Para *i* desde 1 hasta N-1 (ambos incluidos)

Definir el valor de referencia como el valor que hay en la posición *i* (*a[i]*).

Definir un índice *j* que toma el valor *i*-1 (la posición anterior a la que ocupa el valor de referencia).

Mientras ese índice *j* es mayor o igual a 0 Y el valor de la posición con ese índice *j* (*a[j]*) es mayor que el valor de referencia,

Desplazar el valor que hay en la posición *j* a la posición *j*+1

*Decrementar j en una unidad.*

NOTA: El recorrido acaba O BIEN cuando el valor de la posición j es menor que el valor de referencia, O BIEN cuando j es -1. En cualquiera de los dos casos la posición j+1 es la posición h del hueco mencionado antes.

*Fin Mientras*

*Depositar el valor referencia en la posición j+1.*

NOTA: Observad que el valor que hubiera en la posición j+1 al acabar el recorrido ha sido desplazado una posición a la derecha, así que el recorrido genera un hueco en dicha posición para poder depositar el valor de referencia.

*Fin Para*

Ejemplo detallado:

A continuación, se muestra los efectos de implementar el algoritmo de inserción directa sobre el vector desordenado de enteros siguiente: {30, 15, 2, 21, 44, 8}.

| Rec# | Comp        | Acciones                                                                           | a[0] | a[1] | a[2] | a[3] | a[4] | a[5] |
|------|-------------|------------------------------------------------------------------------------------|------|------|------|------|------|------|
|      |             |                                                                                    | 30   | 15   | 2    | 21   | 44   | 8    |
| 1    | 15 con 30   | Comienza j puesto a 0<br>Desplazar a la derecha el valor 30 y decrementar j (a -1) | 30   | 30   | 2    | 21   | 44   | 8    |
| 1    | Final j= -1 | Asignar a a[0] el valor de referencia (15)                                         | 15   | 30   | 2    | 21   | 44   | 8    |
| 2    | 2 con 30    | Comienza j puesto a 1<br>Desplazar valor 30 a la derecha y decrementar j (a 0)     | 15   | 30   | 30   | 21   | 44   | 8    |
| 2    | 2 con 15    | Desplazar valor 15 a la derecha y decrementar j (a -1)                             | 15   | 15   | 30   | 21   | 44   | 8    |
| 2    | Final j=-1  | Asignar a a[0] el valor de referencia (2)                                          | 2    | 15   | 30   | 21   | 44   | 8    |
| 3    | 21 con 30   | Comienza j puesto a 2<br>Desplazar 30 a la derecha y decrementar j (a 1)           | 2    | 15   | 30   | 30   | 44   | 8    |
| 3    | 21 con 15   | Finalizar recorrido puesto que 15 es menor que 21                                  | 2    | 15   | 30   | 30   | 44   | 8    |
| 3    | Final j=1   | Asignar a a[2] el valor de referencia (21)                                         | 2    | 15   | 21   | 30   | 44   | 8    |
| 4    | 44 con 30   | Comienza j puesto a 3<br>Finalizar recorrido puesto que 30 es menor que 44         | 2    | 15   | 21   | 30   | 44   | 8    |
| 4    | Final j=3   | Asignar a a[4] el valor de referencia (44).                                        | 2    | 15   | 21   | 30   | 44   | 8    |
| 5    | 8 con 44    | Comienza j puesto a 4<br>Desplazar 44 a la derecha y decrementar j (a 3)           | 2    | 15   | 21   | 30   | 44   | 44   |
| 5    | 8 con 30    | Desplazar 30 a la derecha y decrementar j (a 2)                                    | 2    | 15   | 21   | 30   | 30   | 44   |

|                                     |           |                                                   |   |         |         |    |    |    |
|-------------------------------------|-----------|---------------------------------------------------|---|---------|---------|----|----|----|
| 5                                   | 8 con 21  | Desplazar 21 a la derecha y decrementar j (a 1)   | 2 | 15      | 21 → 21 | 30 | 44 |    |
| 5                                   | 8 con 15  | Desplazar 15 a la derecha y decrementar j (a 0)   | 2 | 15 → 15 | 21      | 30 | 44 |    |
| 5                                   | 8 con 2   | Finalizar recorrido puesto que 2 es menor que 15. | 2 | 15      | 15      | 21 | 30 | 44 |
| 5                                   | Final j=0 | Asignar a a[1] el valor de referencia (8)         | 2 | 8       | 15      | 21 | 30 | 44 |
| Final de recorridos. Array ordenado |           |                                                   |   |         |         |    |    |    |

Cread un nuevo proyecto llamado “Laboratorio 2 – Arrays”. Cread ahora el package “edu.upc.etsetb.poo.ordenacion”. Cread en ese package la clase OrdenaInsercion. A continuación se muestra el código que debe tener dicha clase:

```
package edu.upc.etsetb.poo.ordenacion ;

public class OrdenaInsercion {

    public static void main (String args[]) {

        /*INSERTAR aquí el código que declara, crea e inicializa convenientemente el array de enteros
        {30, 15, 2, 21, 44, 8}*/

        //INSERTAR aquí el código que muestre por pantalla el contenido del array sin ordenar
        //INSERTAR el código que implementa el algoritmo de inserción directa
        //INSERTAR aquí el código que muestre por pantalla el contenido del array ordenado

    }

}
```

Implementad el algoritmo descrito en el lugar indicado y comprobad su correcto funcionamiento.

**SE RECOMIENDA ENCARECIDAMENTE QUE NO COMENCÉIS REALIZANDO UNA EJECUCIÓN NORMAL.** En lugar de eso definid puntos de parada temporal (breakpoints) que os permitan comparar los contenidos del array con los mostrados en la figura anterior Y ejecutad una sesión de depuración comprobando si los valores intermedios del array coinciden con los de la tabla anterior.

## **Apartado 2: Ordenación por el método de la burbuja (“bubble”)**

El algoritmo consiste en realizar varios recorridos (el primero total y el resto parciales) del array. En cada recorrido se deja ordenada la parte final del array: en el primero, se lleva el elemento mayor a la última posición; en el segundo se lleva el segundo elemento mayor a la penúltima posición, y así sucesivamente hasta que el array queda completamente ordenado.

Cada uno de los recorridos da comienzo en la posición 0. En cada iteración de ese recorrido se compara un elemento del array con el siguiente. Estos elementos se intercambian si no aparecen en el array en el orden requerido. Este proceso se repite hasta que se llega a la zona final del array que ya está ordenada por los recorridos previos.

Los recorridos del array acaban cuando durante la ejecución de uno de ellos no se producen intercambios de elementos ya que la ausencia de intercambios indica que el array ya está ordenado).

Algunas preguntas cuyas respuestas pueden ayudar a generar el código:

- ¿Cuántos bucles se necesitan? ¿Para qué sirve(n)?
- ¿Qué mecanismo será necesario utilizar para guardar constancia de si en un recorrido se han realizado intercambios o no? ¿cómo debe gestionarse este mecanismo?
- ¿Deben ser todos los recorridos desde i=0 hasta n siendo n la dimensión del array?

La figura que sigue muestra los pasos del primer recorrido para un array de 5 elementos. NOTAD QUE TRAS LA EJECUCIÓN DEL PASO 4 EL ARRAY QUEDA DIVIDIDO EN DOS ZONAS: LA ZONA ORDENADA TIENE 1 ELEMENTO (JUSTO EL NÚMERO DE RECORRIDOS QUE ACABAMOS DE EJECUTAR), MIENTRAS QUE LA ZONA NO ORDENADA TIENE 4 ELEMENTOS (JUSTO LA DIMENSIÓN DEL ARRAY MENOS EL NÚMERO DE RECORRIDOS COMPLETADOS).

SUGERENCIA: PROGRAMAD PRIMERO UN RECORRIDO QUE HAGA EXACTAMENTE LO QUE SE MUESTRA EN LA FIGURA QUE SIGUE. UNA VEZ EL PROGRAMA OS REALICE CORRECTAMENTE EL RECORRIDO, AÑADID EL CÓDIGO NECESARIO PARA REALIZAR LOS RECORRIDOS PRECISOS PARA ORDENAR TODO EL ARRAY.

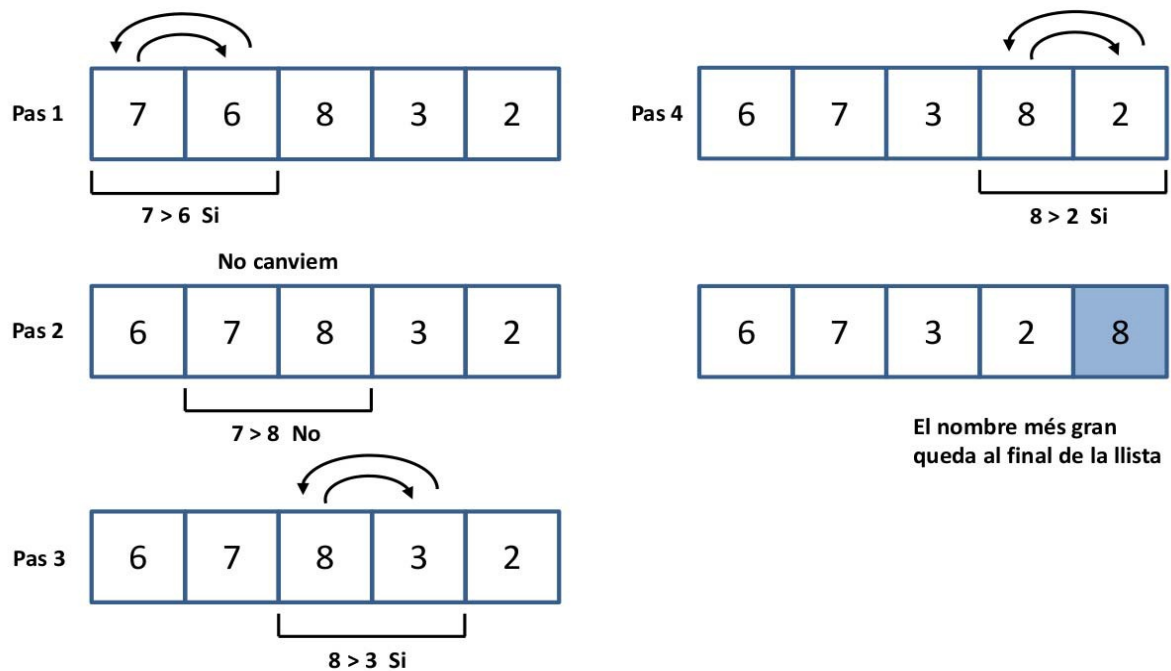


Figura 2. Funcionamiento del algoritmo de la burbuja.

Este algoritmo permite también ordenar el array en sentido decreciente: para ello basta con cambiar la condición de comparación de “mayor que” a “menor que”.

Cread la clase OrdenaBurbuja en vuestro package y proceded como en el caso de la clase OrdenaInsercion. Implementad el código necesario para ordenar el array {7, 6, 8, 3, 2}.

Como antes, SE RECOMIENDA ENCARECIDAMENTE QUE DEPURÉIS LA PRIMERA EJECUCIÓN PARA COMPROBAR QUE EL PROGRAMA REALIZA LAS COMPARACIONES / INTERCAMBIOS ESPERADOS Y QUE LOS VALORES GRANDES SE VAN MOVIENDO HACIA LA DERECHA DEL VECTOR COMO SE ESPERA.

Cuando consideréis que vuestro programa funciona correctamente, probadlo con el array siguiente: {10, 3, 7, 6, 9, 8, 7, 1}. Utilizad entonces el depurador para inspeccionar el contenido del array al final de cada recorrido y el número de intercambios realizados en cada recorrido. Complimentad la tabla que sigue con esa información. Tened en cuenta que el que la tabla tenga 10 filas no implica que haya que realizar 10 recorridos. La tabla se os da con la información del primer recorrido incluida.

| Recorrido | Estado del array al final del recorrido | Número de intercambios realizados |
|-----------|-----------------------------------------|-----------------------------------|
| 1         | { 3, 7, 6, 9, 8, 7, 1, 10 }             | 7                                 |
| 2         |                                         |                                   |
| 3         |                                         |                                   |
| 4         |                                         |                                   |
| 5         |                                         |                                   |
| 6         |                                         |                                   |
| 7         |                                         |                                   |
| 8         |                                         |                                   |
| 9         |                                         |                                   |
| 10        |                                         |                                   |

### **Apartado 3: Ordenación de valores aleatorios**

Cread un nuevo package "edu.upc.etsetb.poo.ordenacion.aleatorio". Copiad en ese package las dos clases construidas en los apartados 1 y 2. Para ello seleccionad una de las clases en la zona de exploración de proyectos. Pulsad el botón derecho del ratón y seleccionad "Copy". A continuación apuntad al package recién creado, pulsad el botón derecho del ratón y seleccionad "Paste"->"Refactor copy". Se os presentará un cuadro de diálogo. Pulsad el botón "Refactor". Netbeans no solo copiará los contenidos de vuestra clase en el nuevo package, sino que cambiará la sentencia de definición del package para que coincida con el nombre del package destino.

A continuación modificad vuestro código para que los contenidos del array a ordenar no vengan predefinidos, sino que en cada ejecución se generen aleatoriamente.

Para ello deberéis utilizar la clase Random de la siguiente forma:

1. Cread un nuevo objeto de la clase Random usando la sintaxis indicada

```
Random random = new Random() ;
```

NOTA: NetBeans os marcará la línea como errónea. A la izquierda de la línea os aparecerá un icono de una bombilla: eso quiere decir que NetBeans tiene al menos una sugerencia de cómo intentar solucionar el problema. En este caso os sugiere importar la clase Random (add import java.util.Random). Seleccionad la primera sugerencia: NetBeans os escribirá la línea

```
import java.util.Random;
```

antes de la primera línea de la definición de vuestra clase. Así tanto el compilador como la máquina virtual de Java saben dónde encontrar la clase Random (no olvidéis que existe una relación directa entre los packages y algunos directorios en el sistema de archivos de la máquina).

2. Programad un bucle que asigne a cada una de las posiciones del vector un valor pseudoaleatorio. Para conseguir ese valor pseudoaleatorio invocad al método cuya cabecera se indica a continuación:

```
public int nextInt(int bound)
```

Este método genera un número entero comprendido entre el 0 (inclusive) y el argumento bound (exclusive).